SESSION 21 / FINAL DEVOPS PROJECT

OpsPulse

Mission control for incidents.



Chhavi Ahlawat · 24BCS10201

Incidents need a shared operational picture

OpsPulse brings impact, ownership and communication into one workflow.

01 **Understand the impact**

See affected services and prioritise by severity.

02 **Coordinate the response**

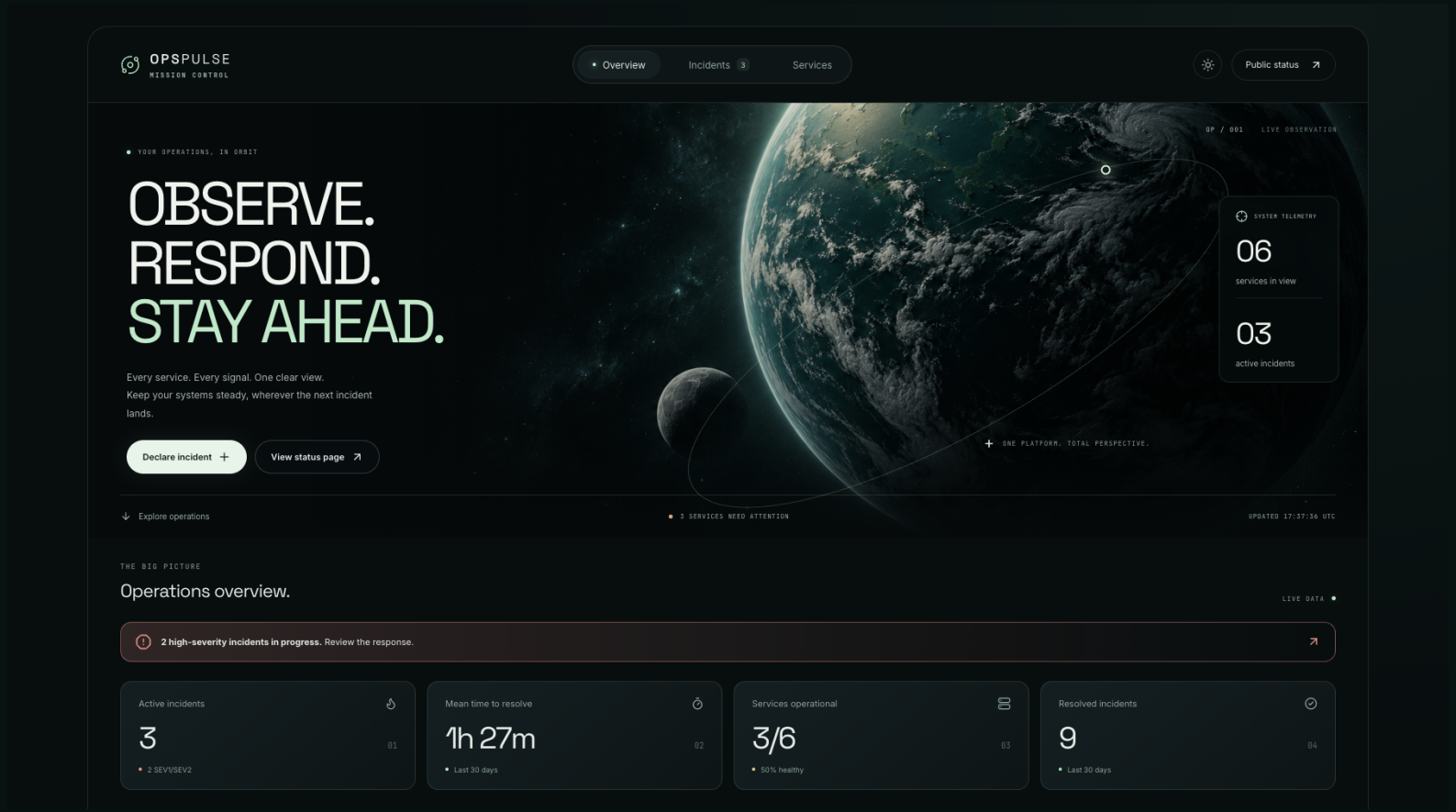
Record ownership, status changes and a response timeline.

03 **Keep users informed**

Publish the current impact and incident history.

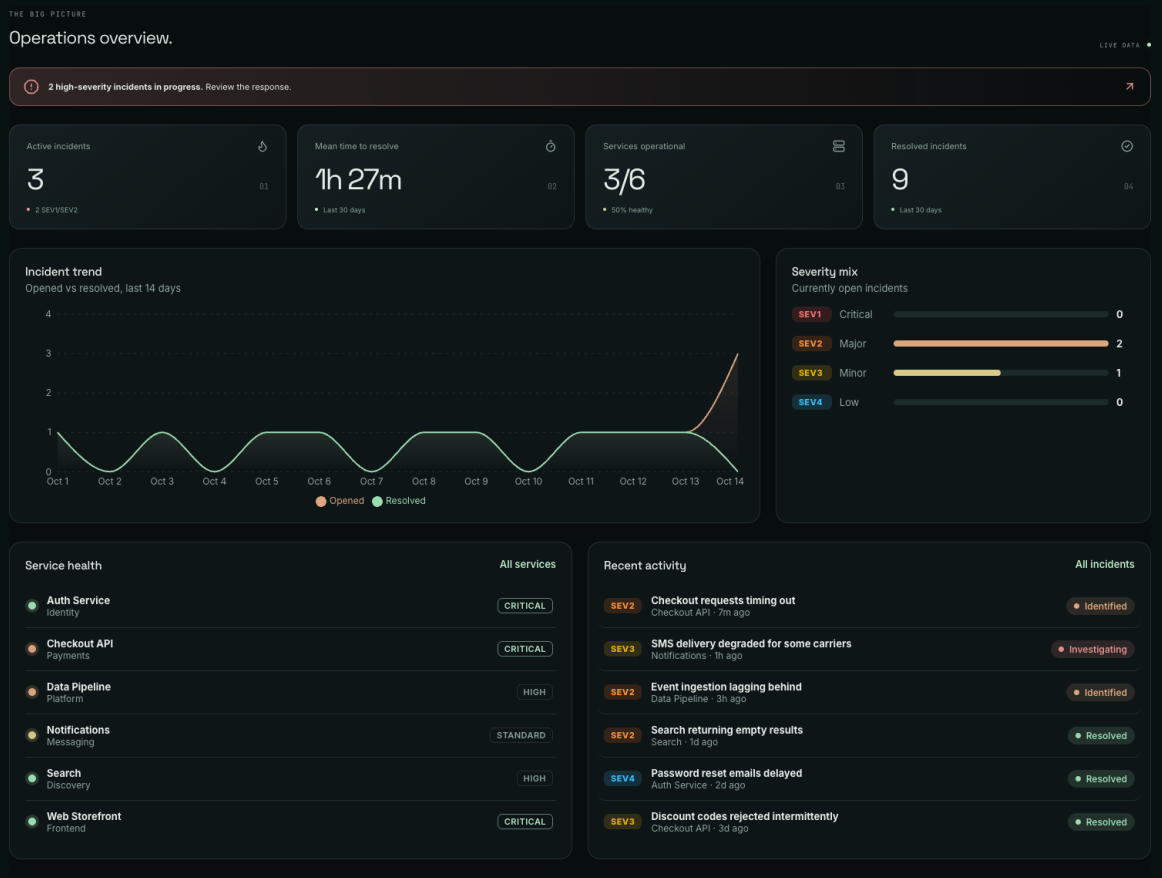
A landing page built around operational clarity

A space-inspired interface with a restrained mint palette.



See impact before opening an incident

The overview combines response KPIs with current service health.



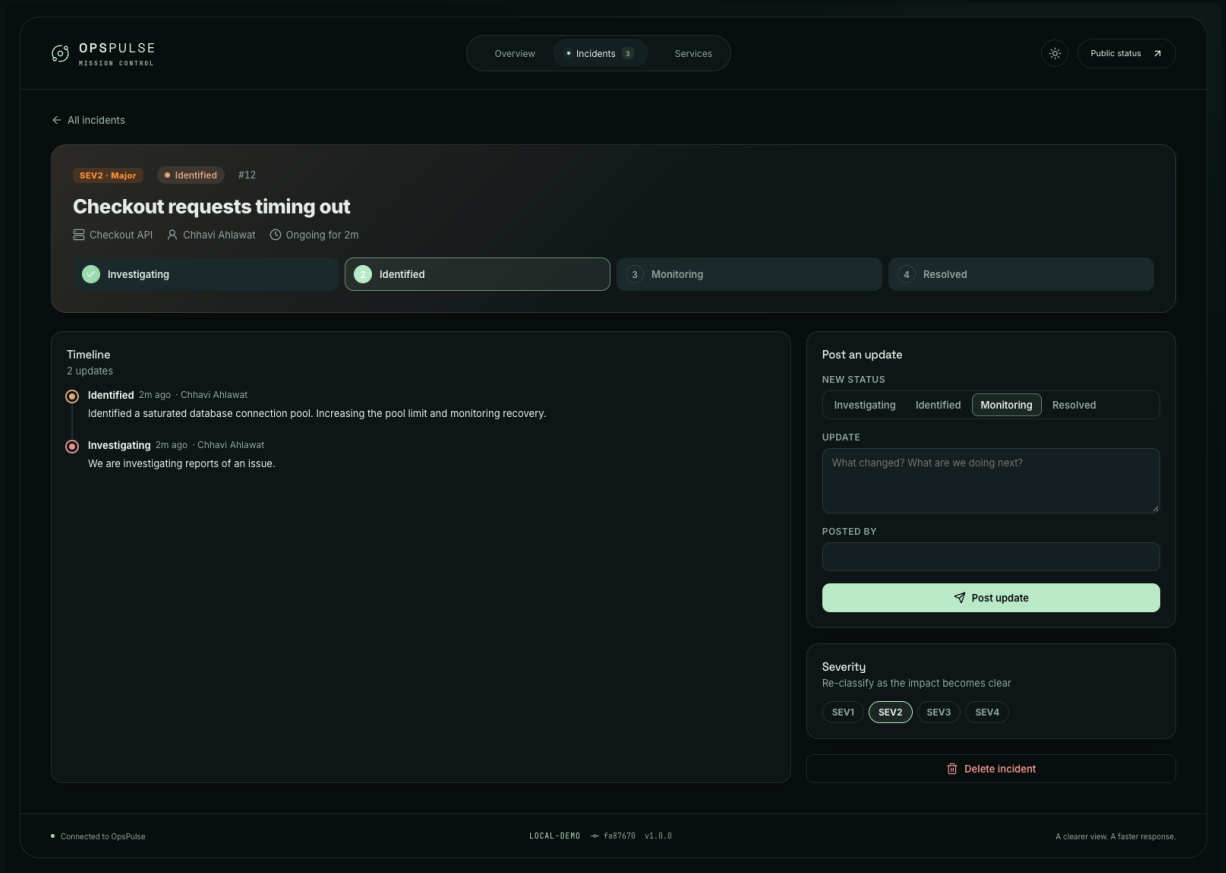
3 active incidents

3 / 6 services operational

Demo data

A clear timeline keeps the response moving

Investigating → Identified → Monitoring → Resolved



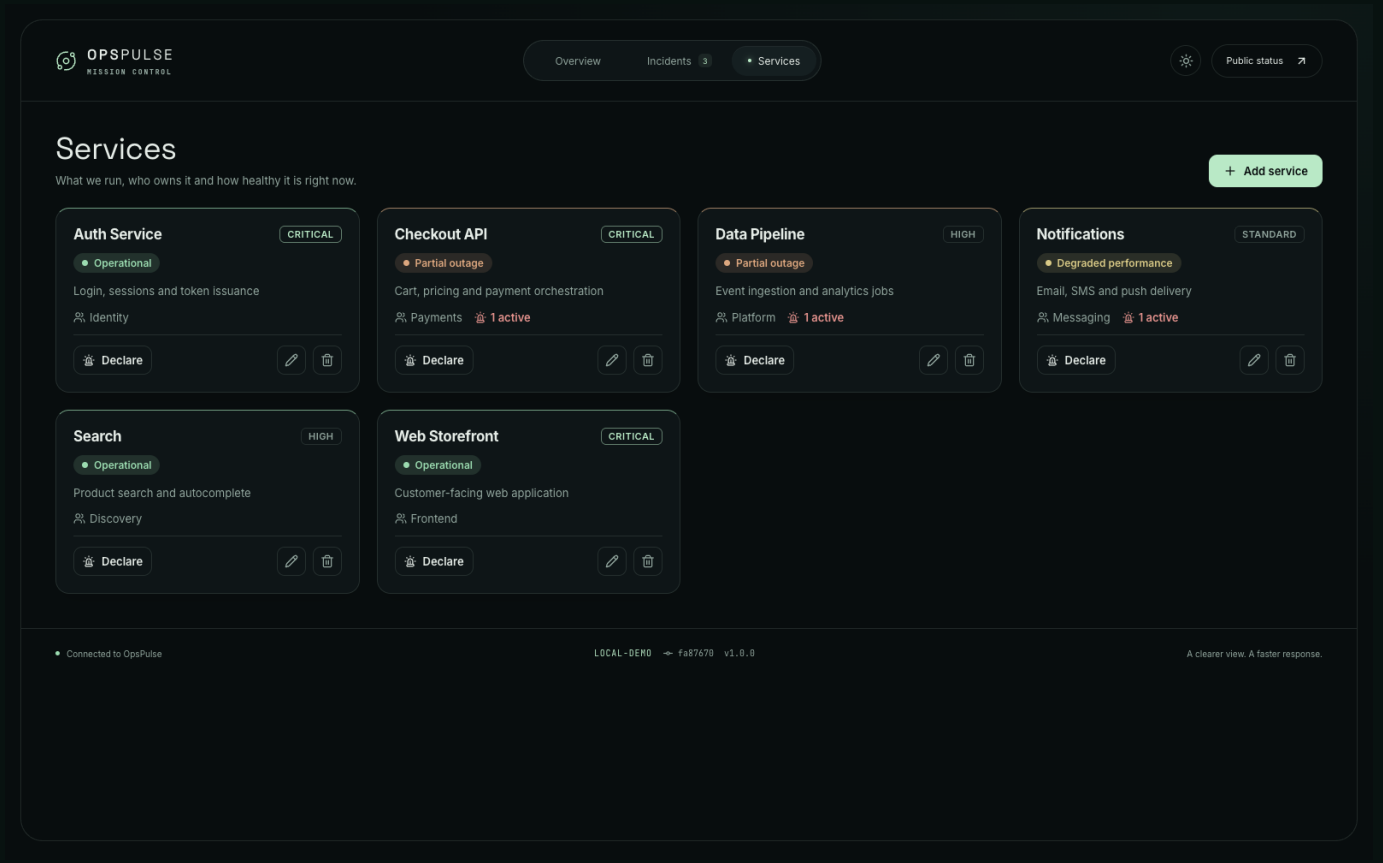
Impact

Ownership

Timestamped updates

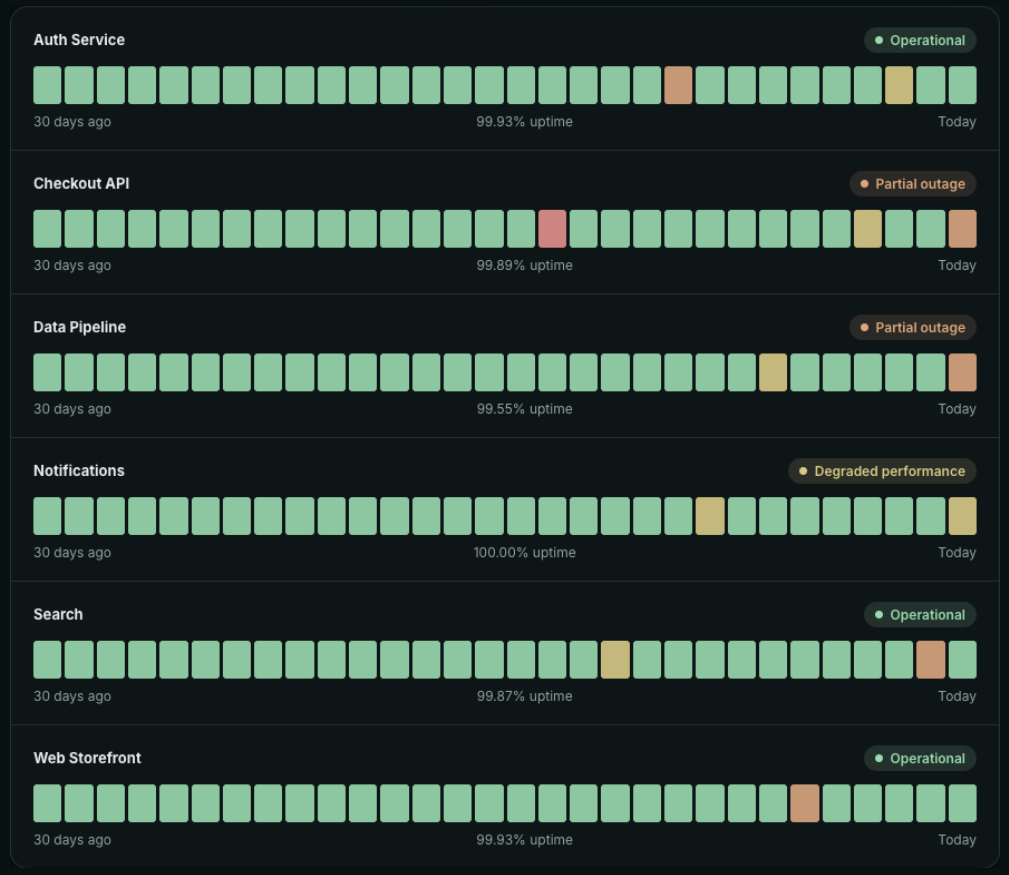
Connect each service to its owner and incidents

The catalogue links service criticality, team ownership and current impact.



Share status without exposing the response console

Thirty-day history is calculated from recorded incidents.



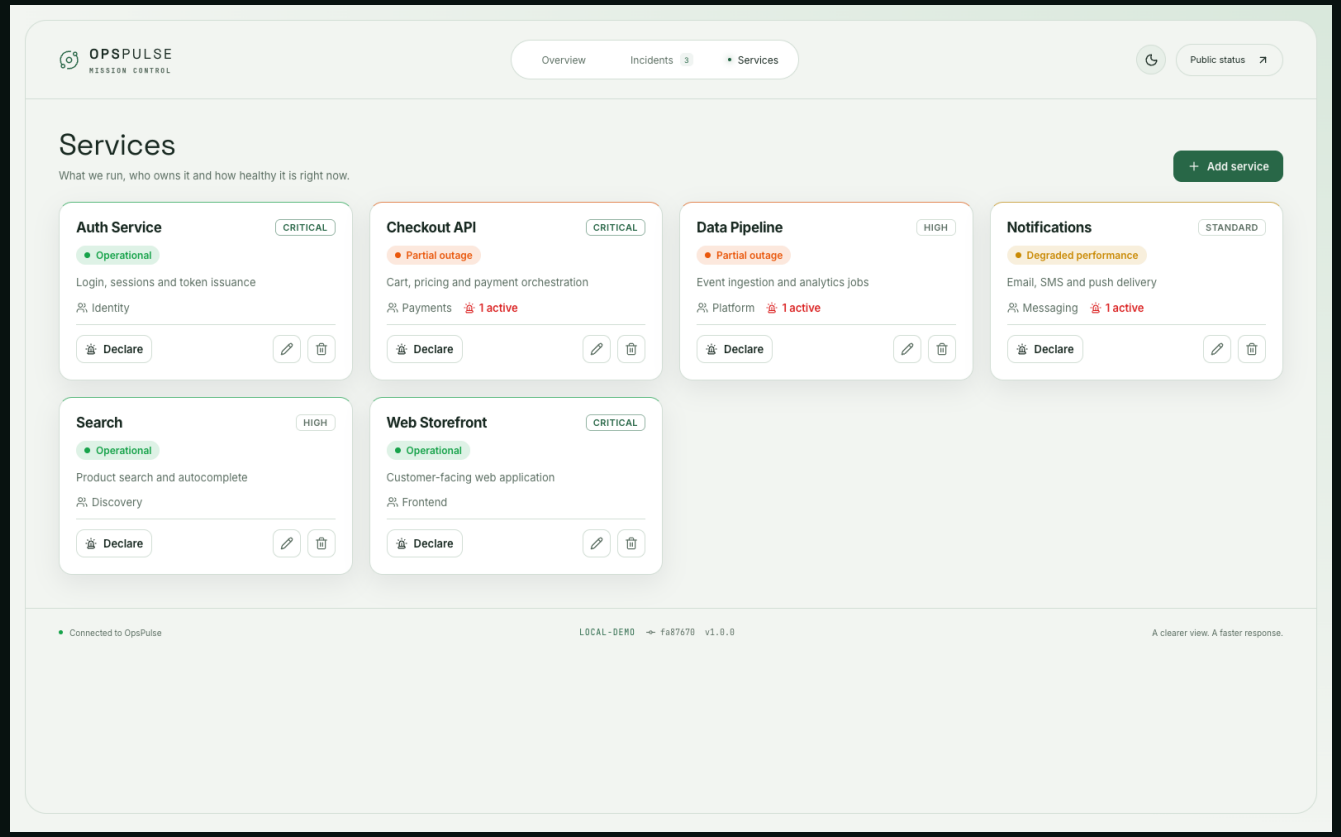
One outage interval, counted once.

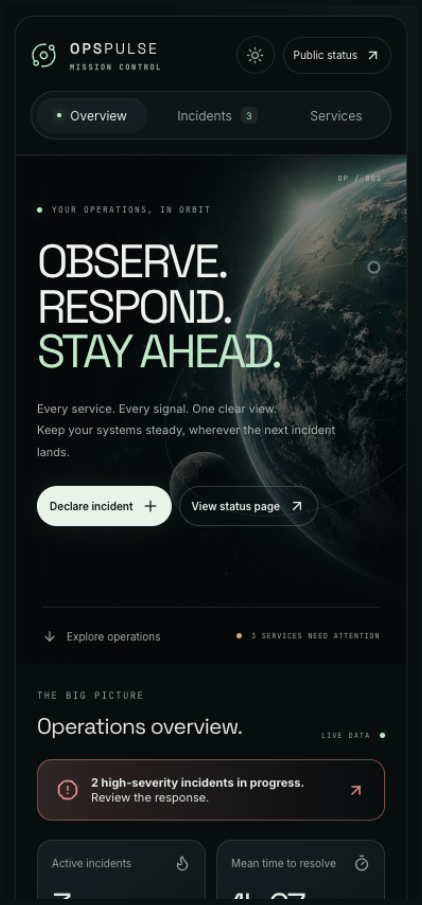
Overlapping outages are merged before uptime is calculated.

These are incident-derived figures, not external probe measurements.

The experience adapts to different screens

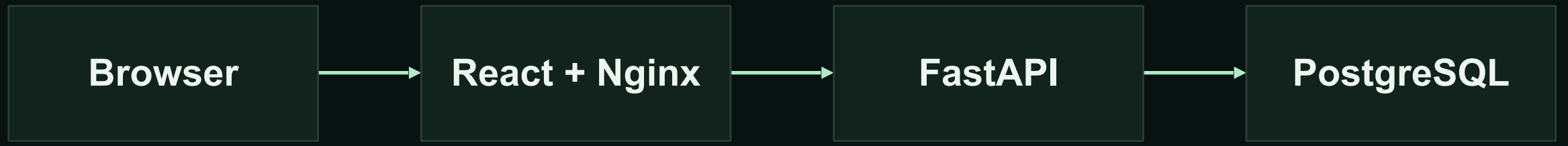
Responsive navigation, light/dark themes and clear feedback states.





Three application layers form one release

React presents the workflow; FastAPI validates it; PostgreSQL persists it.



Service → Incident → IncidentUpdate

Compose and Kubernetes use PostgreSQL. Screenshots use an isolated SQLite demo database.

A documented API keeps behaviour explicit

Interactive OpenAPI documentation is served directly by FastAPI.



/api/services

Catalogue and ownership

/api/incidents

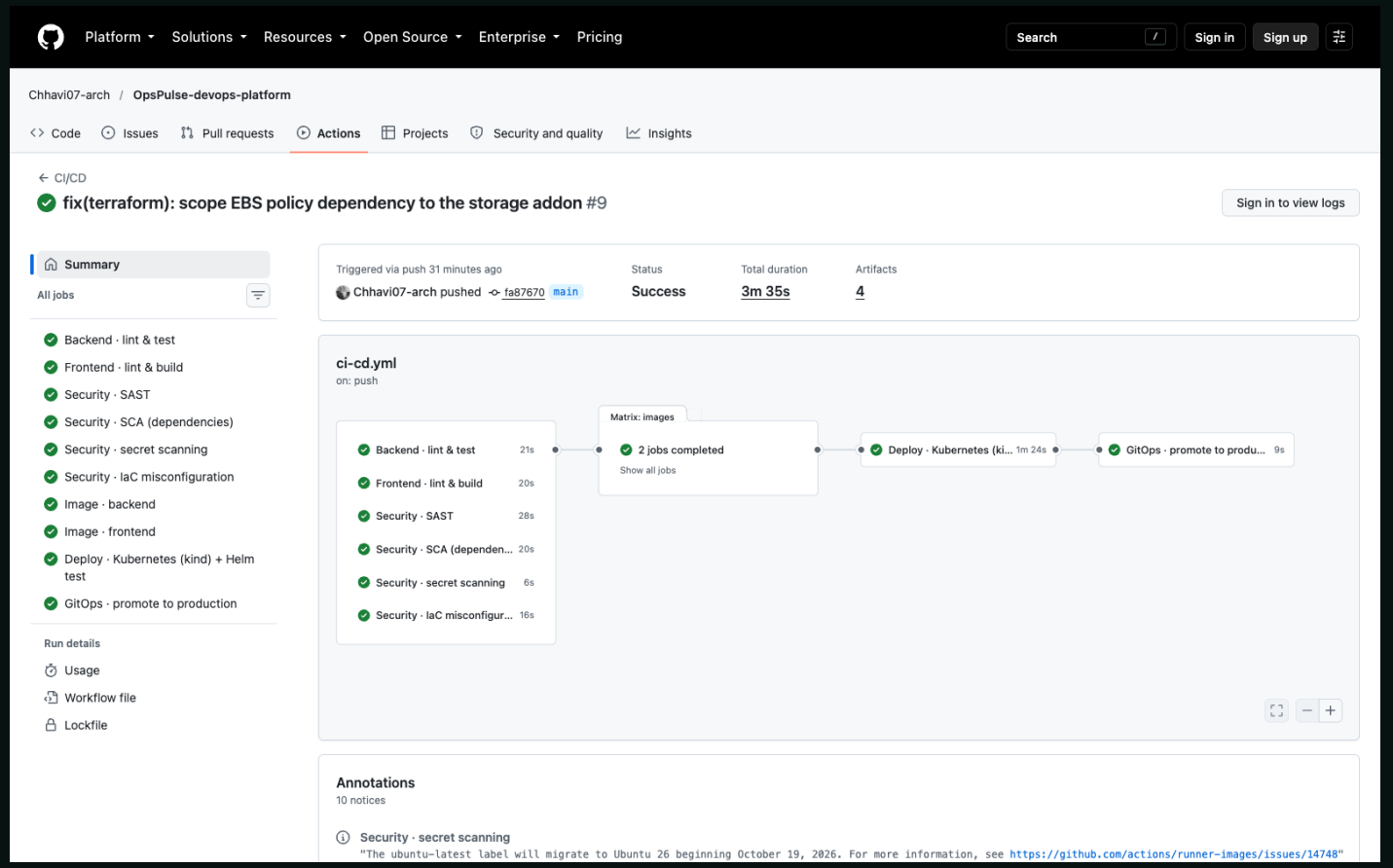
Lifecycle and timeline updates

/health · /ready · /metrics

Process health, database readiness and telemetry

Every release must pass the delivery gates

An actual successful workflow run provides repeatable evidence.



10 jobs passed

Tests → scans → images → kind + Helm → GitOps promotion

Tests check behaviour and deployment readiness

Evidence from the successful CI run for commit fa87670.

30

backend tests passed

97%

backend coverage

Beyond unit tests

Frontend lint/build, Helm installation in kind, and in-cluster endpoint checks.

Security checks run before image promotion

Build-time gates and runtime controls address different failure modes.

Before release

Source: Bandit + Semgrep
Dependencies: pip-audit + npm audit
Secrets: Gitleaks
IaC and images: Trivy

At runtime

Non-root containers
Read-only root filesystems
Dropped Linux capabilities
Restricted PostgreSQL ingress

Demo boundary: write APIs stay private; authentication and RBAC are future work.

Health checks, replicas and storage work together

The Helm chart packages operational behaviour with the application.

Availability	Two frontend and two backend replicas; disruption budgets retain one.
Readiness	Separate process and database checks prevent premature traffic.
Scaling	CPU target 70%; backend 2–6 replicas, frontend 2–4.
Persistence	PostgreSQL StatefulSet + PVC; migrations run with an advisory lock.

A running release can be traced back to Git

Images are identified by the full commit SHA.

01 **Build immutable image tags**

GitHub Actions publishes backend and frontend images to GHCR.

02 **Promote only after the gates pass**

The workflow updates production image tags in Git.

03 **Reconcile the declared state**

Argo CD reads the Helm chart and values, then syncs, self-heals and prunes.

Metrics help explain user-facing symptoms

Prometheus, Grafana and structured logs are configured alongside the app.

Request signals

Rate · errors · latency
HTTP status codes
Pod CPU and memory

Response signals

Active incidents by severity
Declarations and resolutions
Resolution duration

Alerts: unavailable backend, elevated errors, high latency and an open SEV1.

Configuration is implemented; no live monitoring screenshots are claimed.

AWS is prepared for a short, controlled demo

Existing project: Hello World · Sydney: ap-southeast-2

60

resources in the verified plan

EKS 1.35
Two t3.medium workers
Encrypted database volume
Private port-forward access

NOT DEPLOYED

Launch for the demo, then tear down. Total budget: $1–$2.

The hard parts shaped the implementation

Three issues turned into specific engineering decisions.

Concurrent migrations	Use a PostgreSQL advisory lock before changing the schema.
Overlapping outages	Merge intervals so downtime is not counted twice.
Deferred Terraform values	Narrow the EBS policy dependency to the storage add-on.

A working product with a reproducible delivery path

OpsPulse connects incident response to the engineering system that ships it.

Built and verified

Incident workflows and public status
Responsive UI and documented API
30 passing tests and 97% coverage
Security gates + kind/Helm smoke test

Next milestones

Run and tear down the AWS demo
Add authentication and permissions
Integrate notifications
Add external health probes

github.com/Chhavi07-arch/OpsPulse-devops-platform